

★ Member-only story

AI

+

Artificial Intelligence

+

Large Language Models

+

AI Agent

+

Open Source

+

Qwen3.8-Flash-Next: How I Ran The 177B Model on One Gaming GPU — and Why the Secret Isn't the GPU

Qwen3.8-Flash-Next, a filing cabinet, and how I run a 180-billion-parameter model on RTX 3090s I bought years ago



Andrew Zhu

Following ▾

9 min read · Sep 5, 2026



143



5



Every few years, open-source AI drops a model that quietly rewrites the rules of what home hardware can do. Qwen3.8-Flash-Next, Alibaba's August surprise, is one of those.

On benchmarks it beats Qwen3.8-27B, DeepSeek V4 Flash (0731), and even Opus 4.6 on several agentic suites — while activating only **6 billion parameters per token**. I covered the benchmark story in my [previous article](#), so I won't relitigate the scores here.

What I want to talk about is the line that made me sit up in my chair: someone on Reddit claimed they were running this model — a model whose files total **88 gigabytes** — on a **single RTX 3090**.

My first reaction: impossible. A 3090 has 24GB of VRAM. The model is 180 billion parameters. That math is off by an order of magnitude, and I own a calculator.

So I tried it. On my own machine.

It works.

And the reason it works is not brute force, not a magic quantization trick, and honestly, not even the GPU. It's an architectural decision so simple that once you see it, you'll wonder why nobody did it earlier.

Let me show you what's going on — and then give you the exact commands to run this model on one, two, or three RTX 3090s.

What's actually new here

Flash-Next isn't one innovation. It's four, and each one moves some expensive work from expensive hardware to cheap hardware.

1. Ultra-sparse MoE: a hospital on call, not a hospital in every room.

The model has 48 layers, and each layer contains **512 expert sub-networks**. But every token only visits **11 of them: 10** picked by a router that reads the token and selects the best-matched specialists, plus **1 shared expert** — a

generalist that every single token sees, no routing involved. Roughly 2% of the experts do the work for any given word.

Think of it as a hospital network: 512 consultants on call, one general practitioner permanently in the examining room, and each patient (a token) gets ten consultations chosen for its symptoms. The other 501 consultants stay home. Crucially, “stay home” doesn’t have to mean “stay in the GPU” — it just means “be reachable.” Hold that thought; it’s the whole ballgame.

(Why the shared expert? Pure routing kept losing generic knowledge — the generalist holds what every token needs, so the ten picks can specialize harder. It’s in every modern MoE design, and it’s folded into the “6B active parameters” figure.)

2. GDN + QSA: remembering without hoarding.

Normal transformers keep a diary of every token they’ve ever seen (the KV cache), and the diary becomes a mansion at long contexts. Flash-Next mostly doesn’t. Three out of four layers use **Gated DeltaNet**, which compresses history into a small fixed-size state — the diary is a sticky note. The fourth layer uses **Qwen Sparse Attention**, which retrieves from a fixed budget: **512 blocks, 2,048 tokens, no matter whether the context is 32K or 1 million tokens.**

Practical consequence: at 178K context, my KV cache costs about 6GB. A dense model my size would need ten times that. This is why a 24GB card can hold long conversations at all.

3. Gated Residual: widening the highway.

Classic transformers funnel all information through a single “residual stream” — one highway from layer 0 to layer 47, and deep networks watch their early features get diluted in traffic. Flash-Next widens it to four parallel lanes with learned gates controlling what enters and exits each one. During training, one lane spontaneously became a long-range superhighway carrying early information deep into the network. Small change, free capability.

4. The n-gram embedding table: a dictionary bolted onto the model.

51 billion parameters — bigger than the entire rest of the model — that perform **zero matrix multiplications**. They are a lookup table.

This one deserves its own section, because this one is why your 3090 gets to play.

The filing cabinet: 29GB that cost the GPU nothing

Here's how a normal model reads words. Every word becomes an ID number — “dostoevsky” is 41207 — and the model opens a drawer in a lookup table to find what that word means. One drawer per word.

Now consider the phrase: “*dostoevsky wrote the brothers...*”

A single-word drawer doesn't tell the model much about “the.” To know we mean *Karamazov*, the model has to reason it out — using its expensive thinking machinery — every single time, across billions of sentences that mostly say the same thing.

Flash-Next adds a **second filing cabinet** where the drawers are labeled with *phrases*, not words:

```
Drawer "born on october"    → hint: 1985, Honolulu, singer
Drawer "dostoevsky brothers" → hint: Karamazov, novel, 1880
Drawer "def main("          → hint: python entry point
```

While reading, the model hashes the last few words, opens the matching drawer, and gets a pre-chewed hint — for free. These drawers weren't hand-written; during training, every time “born on october” was followed by “8”, that drawer got nudged a little. Over trillions of sentences, each drawer became a distilled average of what usually follows. Roughly **20 million bigram and trigram entries**, injected once, early, at layer 2.

It is memorization of clichés — and most of language is cliché.

Now, the part that should make you lean forward. Compare the two kinds of knowledge in this model:

	Neural network	N-gram table
Work per token	Matrix math (expensive)	Drawer lookup (no math)
Needs the GPU?	Yes, every millisecond	No — CPU can fetch it
Lives in	VRAM	RAM. Even SSD.

And here's the kicker, the property that makes everything else possible: **the drawer address depends only on the input text, not on the model's internal state**. The moment your prompt is tokenized, the CPU knows exactly which

rows the model will need — it can fetch them *while the GPU is still busy on the previous layer*, like a librarian running to the stacks before you've finished the sentence.

So the architecture splits the model across the memory hierarchy by what each tier is good at:

- **VRAM (fast, expensive):** the 6B of parameters that actually compute per token
- **System RAM (medium, cheap):** the expert weights on standby, the 29GB n-gram table
- **NVMe (slow, dirt cheap):** overflow, paged in on demand

The GPU is no longer the library. It's the reading room.

Running it on three RTX 3090s (my now daily driver)

My server: three RTX 3090s (72GB VRAM combined), 48GB DDR4 system RAM, a third-generation NVMe SSD holding the model. Not a workstation. A pile of gaming GPUs I already owned.

The model: unsloth's UD-IQ4_XS GGUF — 88GB across three shards: about 59GB of backbone weights plus the 29GB n-gram table.

You you will need to pull the latest llama.cpp code and rebuild for Qwen3.8-Flash-Next.

```
CUDA_VISIBLE_DEVICES=0,2,3 CUDA_SCALE_LAUNCH_QUEUES=4x llama-server \  
-m /mnt/data_2t/ai_models_all/llm_hf_models/unsloth/Qwen3.8-Flash-Next-GGUF/Qw
```

```
--alias Qwen3.8-Flash-Next \  
--jinja \  
--metrics \  
--host 0.0.0.0 \  
-ngl 99 \  
--batch-size 4096 \  
--ubatch-size 512 \  
--flash-attn on \  
--temp 1.0 --top-p 0.95 --top-k 20 --min-p 0.0 --repeat-penalty 1.0 \  
--split-mode layer \  
--tensor-split 0.9,0.95,1.0 \  
--fit off \  
--main-gpu 0 \  
--ctx-size 178000 \  
--parallel 1 \  
--image-min-tokens 1024 \  
--reasoning-format none \  
--timeout 1200 \  
--ctx-checkpoints 8 \  
--port 8082 \  
--load-mode mmap \  
--cache-type-k q8_0 --cache-type-v q8_0 \  
-ot '^per_layer_token_embd\.weight$=CPU' \  
--reasoning-effort low \  
-t 14
```

Three flags carry all the interesting weight, so let's take them one at a time.

-ot '^per_layer_token_embd\.weight\$=CPU' — this is the n-gram table being evicted from the GPU. The flag says: *any tensor whose name matches this pattern goes to CPU RAM instead of VRAM*. The table's tensor is literally named `per_layer_token_embd.weight`, so the regex catches it and pins its 29GB in system memory. The GPU never reserves a byte for it. When a token needs a drawer, the CPU opens it and hands the hint over — exactly the librarian arrangement from before, expressed in one line of configuration.

--load-mode mmap — memory-mapped file loading. Instead of reading the model into RAM, llama.cpp tells the OS: "here's a file; hand me pages when

someone touches them." The OS then keeps the *hot* parts of the file in whatever page cache is free, and lets the cold parts stay on the SSD. With 48GB of RAM and a 29GB table, this is exactly what I want — the OS becomes my storage bouncer, deciding what deserves to be resident. (On a machine with 128GB RAM I'd switch this to `none` — read everything once, never fault again. More RAM, simpler story.)

`--split-mode layer with --tensor-split` — the 59GB backbone is cut into contiguous chunks of layers across the three cards. All 48 layers run on GPUs (`-ngl 99`), roughly 20GB per card, with the KV cache (quantized to `q8_0`, another quiet miracle: 178K context for ~6GB) fitting alongside.

And the result on hardware I bought for gaming:

Decode: 30–50 tokens/second
Prefill: 400–700 tokens/second
Context: 178,000 tokens

That's a frontier-adjacent model, faster than reading speed, on three used 3090s and a motherboard that fits in a mid-tower.

Running it on ONE RTX 3090 (the honest version)

Okay, you will say — I don't have three RTX 3090s. One is enough — with one condition: **get at least 64GB of system RAM**. The n-gram table (29GB) plus offloaded experts need somewhere to live, and RAM is the cheapest upgrade you can make to an AI rig in 2026.

Same GGUF model, different physics:

```
CUDA_VISIBLE_DEVICES=0 llama-server \
-m /mnt/data_2t_3/ai_models_all/unsloth/Qwen3.8-Flash-Next-GGUF/Qwen3.8-Flash-
--alias Qwen3.8-Flash-Next \
--jinja \
--metrics \
--host 0.0.0.0 \
-ngl 99 \
-ncmoe 38 \
--batch-size 4096 \
--ubatch-size 1024 \
--flash-attn on \
--temp 1.0 --top-p 0.95 --top-k 20 --min-p 0.0 --repeat-penalty 1.0 \
--ctx-size 178000 \
--parallel 1 \
--image-min-tokens 1024 \
--reasoning-format none \
--timeout 1200 \
--ctx-checkpoints 8 \
--port 8083 \
--load-mode none \
-ot '^per_layer_token_embd\.weight$=CPU' \
--reasoning-effort low \
-t 8
```

One new flag does all the work: **-ncmoe 38**.

It means: *keep the expert weights of the first 38 layers in CPU RAM; the remaining 10 layers keep theirs in VRAM*. That's about 43GB of experts living in system memory. Remember the hospital metaphor — the specialists are on call, and being on call doesn't require living inside the operating theater.

Why can we afford to park 43GB of experts in RAM and still get reasonable speed? Sparsity, again. Each token activates 10 of 512 experts per layer, so each token only *touches* about 1.3GB of expert weights. The CPU computes its resident experts locally — no transfer at all — and the GPU gets its resident

experts natively. The penalty is real (the CPU is far slower than a 3090), but it scales with the 2% you touch, not the 100% you store.

This same script works on a 3090, a 4090, or a 5090 — tune only `-ncmoe` to the card:

GPU	VRAM	Suggested <code>-ncmoe</code>	Experts in RAM
RTX 3090	24GB	38	~43GB
RTX 4090	24GB	38	~43GB
RTX 5090	32GB	28-30	~32GB

Open in app ↗

Now, honesty time, because you deserve it more than hype: on the single 3090, decode runs **15–20 tokens per second**.

That is barely usable. It’s fast enough to read along, slow enough that you’ll feel every long generation, and the machine will feel like it’s thinking — because part of it literally is, on a CPU, in system RAM, far from the GPU. For a proof of concept, or a machine that’s already built, it’s a genuinely remarkable party trick: *a 180-billion-parameter model, answering at reading speed, on one gaming card.*

But for daily usage — the model you keep open all day as your working and coding companion — **three or four RTX 3090s are what make it pleasant**. That’s the difference between 18 t/s and 40, between “look what I can run” and “look what I run.”

What's coming next: MTP

One more thing to watch, because it's the sequel to this article.

Flash-Next ships with a trained **MTP head** — a small extra module that predicts the next several tokens at once, so the model can draft ahead and let the big model verify in parallel. Speculative decoding, but with a head trained end-to-end for exactly this job. On vLLM, it measurably delivers **2.5× decode speedups** on real prompts.

As of today, llama.cpp does not yet support this model's MTP head — I checked the source at the current head commit: the architecture is fully supported, but the MTP draft path doesn't include it yet. The plumbing for sibling models is all there, so it's a small patch, not a redesign. I expect the merge within days.

When it lands, my 30–50 t/s becomes plausibly 60–100 — for free, with a software update, on the same hardware.

I'll test it the day it merges, and I'll report the numbers honestly — the good and the bad.

Until then: the biggest models in the world are quietly designing themselves to fit on our machines. That's not just engineering. That's a door opening — and this time, nobody asked for the keys first.

AI

+

Artificial Intelligence

+

Large Language Models

+

AI Agent

+

Open Source

+



Written by Andrew Zhu

4.8K followers · 98 following

Following ▾

on Personal AI | open source contributor | startup founder member | ex-Microsoft | www.linkedin.com/in/andrew-zhu-23407223 | <https://x.com/xhinker>

Responses (5)



Alex Mylnikov

What are your thoughts?



Gmc Oc
Sep 5



Hmmm. IMO the article is a little bait-and-switchy. On the front you talk about running Qwen38-Flash-Next on a single RTX 3090... I was curious, seems implausible to me at a decent context length and decent tps... Then the middle covers how it is... [more](#)



2



1 reply

[Reply](#)



Kamrun Nahar she/her
Sep 13



goodness this was genuinely eye opening



1

[Reply](#)



Julian Dormon

Sep 5



I learn so much from you! Thanks. I wonder how this will run on a mac?



5



1 reply

[Reply](#)

See all responses

